WeddingOS

SYSTEM DESIGN DOCUMENT

Version 1.0 | 9 November 2026

A technical architecture blueprint for the WeddingOS MVP

Prepared from the approved product requirements and collaboratively confirmed architecture decisions.

Document Control

Document	WeddingOS System Design Document
Version	1.0
Date	9 November 2026
Status	Initial architecture baseline
Source	Approved PRD v1.0 and user-confirmed SDD decisions

How to read this document

Confirmed choices are architecture decisions. Deferred/TBD items remain open. No database schema, backup policy, endpoint catalogue or provider quota has been invented.

Table of Contents

1. Document Overview
2. Architecture Principles and Constraints
3. Selected Technology Stack
4. High-Level System Architecture
5. Frontend Design
6. Backend Design
7. Authentication, Sessions and Authorization
8. Wedding Lifecycle and Authorization
9. Media and Gallery Architecture
10. Background Jobs and Email
11. API Contract and Error Handling
12. Security Requirements
13. Deployment and Infrastructure
14. CI/CD and Release Process
15. Testing Strategy
16. Logging, Health and Operations
17. Performance, Capacity and Scalability
18. Product-Specific System Behaviors
19. Deferred Decisions and Open Questions
20. Risks and Mitigations
21. Implementation Phasing

Appendix A. Decision Register

Appendix B. Glossary

1. Document Overview

Purpose: Translate the approved WeddingOS PRD and collaboratively confirmed architecture decisions into an implementation-oriented technical blueprint.

Scope: Architecture, technology stack, frontend/backend organization, authentication, authorization, media and email workflows, deployment, CI/CD, testing, security, operations and initial capacity strategy.

Status: Version 1.0, initial architecture baseline. Items marked Deferred/TBD are intentionally unresolved and must be decided before the affected feature or production launch.

Product summary: WeddingOS is a responsive, English-only web application for collaborative Indian wedding planning. Bride and groom are equal admins; family members can be managers; guests access invitations, RSVP and galleries through secure links without accounts. MVP prioritizes simplicity, familiarity and free-tier-first deployment. AI and payment processing are out of scope.

2. Architecture Principles and Constraints

Principles: Prefer familiar technologies; keep the MVP simple; use a modular monolith; enforce permissions on the backend; protect guest links and uploads; avoid paid infrastructure initially; preserve clear module boundaries.

Constraints: JavaScript throughout; MERN; REST APIs; responsive web only; English only; no AI, payment gateway, UPI/bank integration, vendor accounts, in-app notification centre, real-time collaboration or permanent wedding deletion.

Free-tier requirement: Target zero monthly hosting spend using provider free plans and eligible credits. This is not a guarantee. Verify current eligibility, quotas, terms, bandwidth, storage, IP/networking and email limits. Do not automatically upgrade to paid plans. When a quota is reached, restrict or pause the affected feature with a clear message.

3. Selected Technology Stack

Frontend: React.js + Vite; JavaScript; Tailwind CSS; shadcn/ui; Lucide React; Google Fonts; React Router; Axios; React Context API with useState/useReducer; TanStack Query; React Hook Form; Zod; date-fns and date-fns-tz.

Backend: Node.js + Express.js using JavaScript ES modules. Modular monolith. REST API under /api/v1. Internal layers: routes, controllers, services, repositories and Mongoose models.

Data: MongoDB Atlas with Mongoose. Collections, schemas, relationships and indexes are deferred to a separate database-design phase.

Authentication: Email/password and Google OAuth; JWT access and refresh tokens; secure HTTP-only refresh cookies; bcrypt password hashing.

Media: Cloudinary with direct browser-to-Cloudinary signed uploads. Backend authorizes uploads and validates asset details before persisting metadata.

Email and jobs: Nodemailer + Resend. BullMQ + Redis for background jobs. API and worker are separate processes/containers, initially on the same EC2 host.

Infrastructure: Vercel Free frontend; AWS EC2 with Ubuntu Server LTS for API, worker and self-hosted Redis; Docker + Docker Compose; Nginx; GitHub Actions; Swagger/OpenAPI.

Quality: ESLint, Prettier, Vitest, Jest, React Testing Library, Supertest and Playwright. Exact versions and scripts are implementation details.

4. High-Level System Architecture

Architecture style: One deployable modular-monolith backend. API and background worker are separate runtime processes sharing the codebase and domain modules. Redis provides BullMQ queue storage.

Logical flow: Browser accesses React on Vercel and calls Express over HTTPS. API authenticates/authorizes requests, applies business rules through services, and accesses MongoDB Atlas through repositories/Mongoose. Long-running work is enqueued in Redis and processed by the worker. Media uploads go directly from browser to Cloudinary after the API issues short-lived signed authorization.

Text diagram: Users -> Vercel React/Vite -> HTTPS REST API -> Nginx on AWS EC2 -> Express API container -> Services/Repositories -> MongoDB Atlas. API -> Redis/BullMQ -> Worker container -> Nodemailer/Resend. Browser -> signed authorization -> Cloudinary. API, worker and Redis initially share one EC2 host.

External dependencies: MongoDB Atlas, Cloudinary, Resend, Google OAuth, Vercel and AWS EC2. Vendor business-listing provider is not selected and remains TBD.

5. Frontend Design

Organization: Feature-based architecture. Shared components, layouts, hooks, routes, services and utilities live in common folders; feature-specific UI and logic live inside each feature.

Proposed structure: frontend/src/{components,layouts,features/{auth,dashboard,weddings,events,guests,expenses,vendors,tasks,galleries},hooks,services,routes,utils,assets}.

Routing: React Router separates public, authentication and protected workspace routes. Frontend guards improve UX but are not a security boundary; backend independently authorizes every operation.

State and data: Context API holds small shared state such as authenticated user, current wedding and permission context. useState/useReducer handles local state. TanStack Query manages server state, caching, loading/errors and invalidation. Axios is the HTTP client.

Forms: React Hook Form manages form state; Zod validates client input. Backend independently validates all incoming data using Zod.

UI system: shadcn/ui customized with Tailwind CSS; Lucide React icons; Google Fonts. Exact font family and design tokens are chosen during visual design.

Dates: Use date-fns/date-fns-tz and Asia/Kolkata for display. Store instants consistently in UTC where appropriate; preserve date-only concepts as calendar dates to avoid timezone shifts. Exact persistence convention is part of database design.

6. Backend Design

Modules: auth, users, weddings, wedding-members, invitations, events, guests, expenses, payments, vendors, tasks and galleries. Supporting areas include config, middleware, shared utilities, logging and background jobs.

Layer responsibilities: Routes declare endpoints/middleware. Controllers translate HTTP requests/responses. Services implement business rules. Repositories encapsulate database access. Mongoose models define persistence. Validation schemas reject invalid input. Workers execute asynchronous jobs.

Request flow: HTTP request -> route middleware (authentication, authorization, rate limit, validation) -> controller -> service -> repository/model -> MongoDB. Errors flow to centralized error middleware.

API style: REST over HTTPS with JSON and /api/v1 versioning. Detailed endpoint catalogue, payload schemas and endpoint authorization matrix are deferred to API design.

7. Authentication, Sessions and Authorization

Methods: Email/password and Google OAuth. Email/password accounts require email verification. Provide forgot-password and reset-password flows.

Password storage: Use bcrypt with configurable work factor. Never store plaintext passwords. Password reset uses secure, time-limited, single-use tokens; exact token persistence/expiry TBD.

Session model: Short-lived JWT access tokens and longer-lived refresh tokens in Secure, HttpOnly cookies. Implement refresh rotation and server-side revocation/session tracking so removed managers lose access immediately. Exact lifetimes, cookie domain, SameSite, CSRF and session-record design remain TBD.

Google OAuth: Restrict client/server redirect URIs to approved environments. Keep private OAuth secrets server-side.

Authorization: RBAC for Admin and Manager workspace users. Guests have no accounts and use purpose-specific opaque tokens. Backend middleware and services enforce permissions and wedding lifecycle state. Frontend guards are convenience only.

Role rules: Bride and groom are equal admins. Managers have PRD-defined operational access but cannot manage admins, delete expenses/gallery content or control lifecycle. Removing a manager revokes active access immediately; historical records remain and pending tasks are reassigned by an admin.

Guest links: Generate cryptographically random unpredictable tokens; store only hashes. Guest invitation links remain valid until the family record is deleted. Use separate purpose-specific tokens for invitation viewing and gallery uploading.

Manager invitations: Random tokens, hash at rest, seven-day expiry, single use and invited-email verification. Pending invitations have no manual revocation control per PRD.

8. Wedding Lifecycle and Authorization

States: Active, Completed and Archived. Enforce state transitions and all write restrictions on the backend.

Completed: Both admins confirm. Workspace becomes read-only for normal operations; pending tasks are system-completed. Invitations, event information and galleries remain accessible; uploads continue for admins, managers and guests.

Reopening: Both admins approve reopening a completed wedding.

Archived: Both admins approve archiving. Workspace is read-only; admins/managers can view; guest invitations and galleries remain accessible; all uploads are disabled.

Restoration/deletion: Both admins approve restoration. Permanent wedding deletion is not allowed.

Enforcement: Lifecycle checks apply to API services, background jobs and upload authorization. UI hiding/disablement is not sufficient.

9. Media and Gallery Architecture

Upload flow: Browser requests authorization. API authenticates user or validates guest upload token, checks role and lifecycle, and creates short-lived Cloudinary signed parameters. Browser uploads directly to Cloudinary. API validates returned asset details and persists metadata.

Permissions: Admins/managers upload in Active and Completed; guests upload through event upload links in those states. Archived weddings reject uploads. Managers cannot delete. Admins can delete individual/bulk items. Guests can view/download individual files and upload through upload links.

Separate links: Separate view and upload links. QR codes open upload links. Guest upload links are bearer credentials; make them unguessable, scoped and rate-limited. View permission does not imply upload permission.

Validation: Validate permission before issuing signatures. Restrict MIME/resource types and size; validate Cloudinary response, folder/resource type and ownership. Do not trust client-provided URLs or metadata. Exact limits TBD.

Storage and behavior: Cloudinary stores binaries; MongoDB stores metadata. No visible per-wedding allowance, but technical safeguards and provider quotas apply. Per-event Photos/Videos tabs; galleries remain viewable. No guest albums, uploader identity, approval, ZIP or bulk guest download.

10. Background Jobs and Email

Queue: BullMQ backed by Redis. API enqueues work and responds without waiting for bulk email. Separate worker processes jobs.

Use cases: Guest event-change/cancellation emails, manager and guest invitations, account verification, password reset and approved transactional emails.

Provider: Nodemailer composes/sends through Resend. Credentials remain server-side. Configure verified sender identity where available.

Reliability: Jobs should be idempotent where practical, use bounded retry/backoff and record terminal failures. Avoid duplicate emails on request retries. Exact job schemas, concurrency, retry counts and retention TBD.

Free-tier behavior: Monitor provider quotas. Do not auto-upgrade. Queue/pause eligible sends and communicate limits; never claim delivery success before provider acceptance.

11. API Contract and Error Handling

Response format: Standard JSON envelope. Success: { success: true, message: '...', data: {...} }. Error: { success: false, message: '...', errors: [{ field: '...', message: '...' }] }. Exact envelope finalized during implementation.

Status codes: Use appropriate codes: 200/201 success, 204 no body, 400 invalid input, 401 unauthenticated, 403 unauthorized, 404 missing resource, 409 conflict, 500 unexpected server error.

Pagination: Offset/page-based initially, e.g. GET /api/v1/guests?page=1&limit=20. Return page, page size, total records and total pages; enforce maximum page size.

Search: Advanced search-engine design deferred. Basic queries required by product features may use MongoDB queries.

Documentation: Swagger UI generated from OpenAPI. Document methods, parameters, schemas, authentication, role restrictions and errors.

12. Security Requirements

Transport/browser: HTTPS in production. Explicit CORS allowlist per environment. CORS is not authentication. Configure secure headers, cookie flags and CSRF protection for cookie-authenticated refresh operations.

Requests: Backend Zod validation, input normalization/sanitization, body limits and express-rate-limit for login, registration, password reset, invitation, RSVP, public-link and upload-authorization endpoints.

Authorization: Check wedding scope on every protected data access; prevent IDOR; enforce role and lifecycle rules in services.

Tokens: High-entropy random guest/manager tokens, hashes at rest, narrow scope, expiry/consumption rules and revocation. Never log tokens or secrets.

Media: Short-lived signed Cloudinary authorization, strict types/size limits, server-side verification and scoped upload tokens. Never expose Cloudinary secret in browser.

Secrets: Local .env, committed .env.example placeholders, GitHub Actions Secrets and restricted production configuration. Keep .env out of Git. Never expose private credentials via Vite frontend variables.

Operations: Restrict SSH, keep Redis private, minimal EC2 security groups, update OS/images/dependencies and use non-root containers where practical.

13. Deployment and Infrastructure

Frontend: Vercel Free, initially using a provider subdomain such as weddingos.vercel.app subject to availability.

Backend: AWS EC2 Ubuntu Server LTS. Docker Compose runs API, worker and private Redis containers. Nginx reverse-proxies API traffic. Exact instance, region, network and certificate setup TBD.

External services: MongoDB Atlas for data; Cloudinary for media; Resend for email. Each has independent free-plan quotas and terms.

Containers: Docker images for API and worker; Redis private container. Docker Compose for local development/staging and initial EC2 deployment. Configure health checks and restart policies.

Environments: Development local; Staging local via Docker Compose with separate credentials; Production on Vercel + EC2. Separate environment variables/database credentials.

Domain/HTTPS: Free provider subdomains initially; no purchased custom domain required for MVP. Final backend hostname, certificate and cookie domain policy must be designed together.

Free-tier safeguards: Verify current Vercel, AWS, Atlas, Cloudinary and Resend eligibility/limits before launch. Storage, outbound bandwidth, public IPv4, snapshots, transfer, domain and over-quota usage may cost money. Pause/restrict rather than auto-upgrade.

14. CI/CD and Release Process

Repository: One GitHub monorepo containing frontend, backend, Docker Compose, docs and GitHub Actions.

CI: On PRs/pushes install dependencies, run lint/format checks, frontend/backend unit tests, API tests and builds. Playwright E2E runs in an appropriate test pipeline.

Deployment: After checks succeed on approved branch, deploy frontend to Vercel and backend/worker to EC2. Store credentials in GitHub Secrets and run post-deploy health checks.

Release safety: Stop deployment on failed checks. Keep rollback path to previous known-good revision. Database migration/rollback policy deferred until schema design.

15. Testing Strategy

Frontend: Vitest and React Testing Library for unit/component behavior, forms, route guards and state transitions.

Backend: Jest and Supertest for services, validation, middleware, permissions, lifecycle checks and response contracts.

End-to-end: Playwright for sign-in, wedding creation, event management, guest invitation/RSVP, expense recording, manager restrictions and gallery access.

Critical cases: Equal Admin controls; Manager restrictions; immediate manager revocation; single-use seven-day manager invitations; guest link invalidation on deletion; one RSVP per family/event; expense/payment calculations; lifecycle transitions; archived upload rejection; email retry behavior.

Test data: Use isolated test data/credentials. Never use production personal data in automated tests.

16. Logging, Health and Operations

Logging: Pino or Winston (final choice TBD) for structured logs; distinguish API and worker logs. Docker logs provide initial collection. Never log passwords, tokens, cookies or unnecessary personal data.

Errors: Centralized Express error middleware and consistent response. User-safe messages; diagnostic detail in server logs. No stack traces or database internals in production responses.

Health: Liveness/readiness endpoints without sensitive details, Docker health checks and post-deployment verification.

Monitoring: Lightweight CPU, memory, disk and process checks. No paid monitoring in MVP. Runbooks for API/worker failure, Redis unavailable, email quota exhaustion and Cloudinary upload failures.

17. Performance, Capacity and Scalability

Initial target: 10-50 weddings initially, up to 500 guest families per wedding and a small number of simultaneous users. These are planning targets, not hard product limits or guarantees.

Scaling: Vertical scaling first: optimize code/queries, add indexes after database design, increase EC2 resources when needed. Keep modular monolith. Add separate hosts/horizontal scaling only when measured demand justifies it.

Practices: Paginate large lists, avoid unbounded queries, keep media binaries out of MongoDB, use Cloudinary transformations where appropriate, process bulk email asynchronously and cache server data with TanStack Query.

Single-host limitation: API, worker and Redis share one EC2 host; host failure affects all backend services. Monitor resources and limit worker concurrency so email processing does not starve interactive API traffic.

18. Product-Specific System Behaviors

Dashboard/events: Admins and managers share dashboard with upcoming events, tasks, expense and RSVP summaries. Events support custom/predefined templates, multi-day schedules, private/guest-facing fields, cancellations and guest email updates. WhatsApp sharing is manual prefilled text only.

Guests: Manual entry and CSV/Excel import with preview, validation and duplicate suggestions; no automatic merge. One family-level record and one RSVP per family/event. No guest login/OTP.

Expenses: Record keeping only; no payment processing. Track agreed amounts and separate instalments. Admins alone delete/restore expenses. Vendor financial records link to expenses to avoid double counting.

Vendors: Manual vendor records and public business listing search by category and manually entered city/location. Listing data provider TBD. No vendor accounts/booking portal.

Tasks: Every task belongs to an event; multiple assignees; Pending/In Progress/Completed. Any assignee can complete. Admin reassigns pending tasks when a manager leaves. Completion system-completes pending tasks.

Lifecycle: Active, Completed and Archived rules in Section 8. Permanent wedding deletion prohibited.

19. Deferred Decisions and Open Questions

Database: Collections, document shapes, references/embedding, indexes, unique constraints, transaction boundaries, retention and migrations deferred.

Backup/recovery: Frequency, destination, retention, restore tests and recovery objectives deferred. This is a production-readiness gap and must be resolved before storing important live wedding data.

API details: Endpoint list, request/response schemas, pagination per resource, error catalogue and role/action matrix deferred.

Provider settings: Verify AWS eligibility, free-tier/credit duration, EC2 options, region and IP/network charges; verify Vercel, Atlas, Cloudinary and Resend quotas.

Session settings: Set exact access/refresh lifetimes, rotation/reuse detection, cookie policy, CSRF protection, session revocation and reset-token rules.

Media/email: Set media types/sizes/concurrency, Cloudinary folder and quota behavior; email sender, templates, delivery tracking, retry policies and quota exhaustion behavior.

Vendor listings: Select compliant business-listing provider or define manual-only discovery for MVP.

Production readiness: Complete backup/recovery, incident runbooks, dependency update cadence, security review and production acceptance checklist.

20. Risks and Mitigations

Free-tier variability: Plans/limits change and may not support production traffic. Verify current terms, monitor usage and degrade gracefully.

Single EC2 host: API, worker and Redis share a failure domain. Use health checks, restart policies, conservative resources and recovery runbooks; backup plan remains prerequisite.

Bearer-link forwarding: Links can be forwarded. Use high entropy, hash-at-rest, narrow scope, rate limits, separate view/upload tokens and invalidation rules.

Cross-origin cookies: Vercel and EC2 hostnames differ. Finalize domains, SameSite/CSRF/CORS and test browsers across environments.

Email/media quotas: Bulk email or media may exceed free quotas. Queue jobs, validate uploads, monitor usage and pause rather than auto-upgrade.

Authorization complexity: Roles, lifecycle and public links create access-control risk. Centralize backend policy checks and test role/lifecycle matrices.

21. Implementation Phasing

Phase 1 - Foundation: Monorepo, Vite/React, Express, Docker Compose, lint/format, env config, health checks, centralized errors and CI.

Phase 2 - Identity/workspace: Email/password, verification, Google OAuth, access/refresh sessions, wedding creation, roles, manager invitations and revocation.

Phase 3 - Core planning: Events, guests, invitations, RSVP, tasks, expenses, payments and dashboard.

Phase 4 - Vendors/galleries: Vendor records/listing-provider decision, Cloudinary signed uploads, gallery viewing/download and lifecycle-aware permissions.

Phase 5 - Notifications/release: BullMQ/Redis worker, Resend transactional email, OpenAPI docs, Playwright critical journeys, staging and production deployment.

Phase 6 - Production readiness: Database schema/index review, backup/restore plan, quota/cost checks, security review, runbooks and acceptance testing before real users.

Appendix A. Decision Register

Confirmed: MERN; Vite; Tailwind; React; Node/Express; MongoDB Atlas/Mongoose; JWT access+refresh; Google OAuth; Cloudinary; Nodemailer+Resend; BullMQ+Redis; REST; modular monolith; feature-based frontend; Context API; RBAC + guest opaque tokens; signed direct uploads; Docker; EC2 Ubuntu LTS; Vercel frontend; free-tier-first; Express rate limiting; lightweight logs; comprehensive tests; GitHub Actions; Swagger/OpenAPI; .env + GitHub Secrets; React Router; Axios; React Hook Form + Zod; backend Zod; bcrypt; shadcn/ui; Lucide; TanStack Query; date-fns; controller/service/repository; standard JSON responses; offset pagination; ESLint/Prettier; JavaScript; monorepo; three environments; local staging; small MVP; vertical scaling.

Deferred: Database schema/indexes; backups; detailed API contracts; advanced search; exact token lifetimes/cookie/session policy; exact provider quotas/resource sizing; vendor listing provider; media limits; email templates/retry policies; final fonts; logging library.

Appendix B. Glossary

RBAC: Role-Based Access Control: authorization based on user roles and assigned permissions.

Opaque token: Random identifier whose meaning is stored server-side; it does not encode claims like a JWT.

Modular monolith: One application/deployment organized into well-separated business modules.

BullMQ: Node.js job queue library using Redis for background work.

CORS: Browser policy controlling which origins may read API responses; not authentication.

ODM: Object Document Mapper; Mongoose maps application objects to MongoDB documents.

CI/CD: Continuous Integration and Continuous Delivery/Deployment: automated checks and release workflows.